

Build Task — Project 1: Beginner LLM App — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're combining everything from the 5 practice exercises above into one small chatbot: memory (so it remembers earlier turns), streaming (so replies feel instant), and safe handling of the failure types you already saw (bad key, too-long input, a structured-output request that doesn't come back clean).

Think of it as 3 files working together: one that talks to the model (`chat_client.py`), one that describes the shape of any structured answer you want back (`schemas.py`), and one that runs the actual terminal loop (`main.py`).

Here are the exact pieces you need to look up and use:

- `load_config()` and `get_logger()` from Doc01 — call them at the very top of `main.py` .
- The multi-turn `history` list pattern from the conversation-memory exercise.
- The streaming loop from the streaming-replies exercise.
- A Pydantic `BaseModel` for the structured-output shape.
- `try/except` blocks for `openai.AuthenticationError` and `openai.BadRequestError` , one each — not one shared catch-all.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The Build Task is a synthesis, not new material — every function here is a small variation on something you already built in the 5 exercises above. `chat_client.py` should expose `create_client()`, `send_message(history, user_input) -> str`, and `stream_message(history, user_input)`, so `main.py` never touches the OpenAI SDK directly — it only calls your own, already-tested wrapper functions.

Reusing Doc01's `config.py` / `logging_setup.py` here isn't optional decoration — it's the whole point of building them in Doc01: this is the first of many projects that starts by importing them unchanged.

- `from config import load_config` / `from logging_setup import get_logger` — same functions from Doc01, unchanged, called at the top of `main.py`.
- `history: list[dict]` — the growing message list; every user turn and assistant reply gets appended to it, in order.
- Streaming: reuse `stream_message()`'s loop, but have it also return the accumulated `full_reply` string so `main.py` can append it to `history`.
- `schemas.py`: `class ExtractedInfo(BaseModel): ...`. Use `client.beta.chat.completions.parse(response_format=ExtractedInfo, ...)` to get back an object Pydantic has already validated, instead of a raw string you'd have to parse and hope is valid JSON.
- Separate `except` blocks matching the "Handles gracefully" requirements: `openai.AuthenticationError`, `openai.BadRequestError` (context length), and a Pydantic `ValidationError` for a structured-output reply that didn't come back clean.

The one genuinely new design question the Build Task raises on its own: **what should happen to `history` after a `BadRequestError`?** Losing the whole conversation and starting over is safe but harsh. Silently dropping the oldest turns and continuing (as sketched below) keeps the conversation alive, at the cost of the model "forgetting" the earliest parts — a real trade-off, not a free fix.

```
except openai.BadRequestError:
    logger.warning("Conversation exceeded context length, trimming history.")
    print("That got too long – forgetting the oldest messages and continuing.")
    history = [history[0]] + history[-4:]
```

Sketch the rest of `main.py`'s loop — the streaming call, the structured-output branch, and both `except` blocks — before checking Hint 2.

Difference between Basic and Intermediate: Basic describes the pieces in plain words.

Intermediate wires them into typed `chat_client.py` / `schemas.py` / `main.py` files and makes the one Build-Task-specific decision none of the individual exercises could make on their own — what to do with `history` the moment it goes over the limit.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
main.py:
    load config and logger from Doc01

    make the chat client
    start history with a system message

    loop forever:
        get user input
        try:
            stream the reply, show it as it arrives
            add both turns to history
        except bad key:
            print a clean message, stop
        except too long:
            print a clean message, drop the oldest turns, keep going

    (separately, on request) call the structured-output mode:
        try:
            get a checked object back
        except it doesn't validate:
            print a clean "couldn't understand that" message
```

The trickiest part — the structured-output call and its own specific error handling:

```
from pydantic import BaseModel, ValidationError

class ExtractedInfo(BaseModel):
    name: str
    intent: str

def get_structured_reply(client, user_input):
    try:
        completion = client.beta.chat.completions.parse(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": user_input}],
            response_format=ExtractedInfo,
```

```
)  
    return completion.choices[0].message.parsed  
except ValidationError:  
    return None # caller should print a clean "couldn't understand that" mes
```

Expected output if you run just this (nothing calls it yet): nothing — try wiring the rest of `main.py` 's loop yourself before checking the Solution.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
chat_client.py:
    def create_client() -> OpenAI: ...
    def stream_message(client, history, user_input) -> str: ...    # prints as it

schemas.py:
    class ExtractedInfo(BaseModel): ...

main.py:
    config = load_config()
    logger = get_logger(__name__)
    client = create_client()
    history = [{"role": "system", "content": "..."}]

    loop:
        user_input = input("You: ")
        try:
            stream_message(client, history, user_input)
        except openai.AuthenticationError:
            logger.error(...); print clean message; stop
        except openai.BadRequestError:
            print clean message; history = [history[0]] + history[-4:]
```

```
from pydantic import BaseModel, ValidationError
from openai import OpenAI

class ExtractedInfo(BaseModel):
    name: str
    intent: str

def get_structured_reply(client: OpenAI, user_input: str) -> ExtractedInfo | None:
    try:
        completion = client.beta.chat.completions.parse(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": user_input}],
            response_format=ExtractedInfo,
        )
        return completion.choices[0].message.parsed
```

```
except ValidationError:  
    return None
```

Wire this into `main.py` behind its own command (e.g. the user types `/extract ...`), with its own `if result is None:` branch printing a clean failure message. Try finishing the rest yourself, then compare against the [Solution](#).

Difference between Basic and Intermediate: Basic's pseudocode and near-complete code prove the structured-output piece works in isolation. Intermediate wires that same piece into a typed `chat_client.py` / `main.py` structure with the 2 required `except` blocks.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Read both depths — they're not "wrong, right," they're 2 real, valid stages of building the same app, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# chat_client.py
from openai import OpenAI

def create_client():
    return OpenAI()

def stream_message(client, history, user_input):
    history.append({"role": "user", "content": user_input})
    stream = client.chat.completions.create(model="gpt-4o-mini", messages=history)
    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="")
            full_reply += piece
    print()
    history.append({"role": "assistant", "content": full_reply})
    return full_reply
```

```
# main.py
import openai
from config import load_config
from logging_setup import get_logger
from chat_client import create_client, stream_message

config = load_config()
logger = get_logger("main")
client = create_client()
history = [{"role": "system", "content": "You are a helpful assistant."}]
```

```
while True:
    user_input = input("You: ")
    try:
        stream_message(client, history, user_input)
    except openai.AuthenticationError:
        print("Login failed – check your API key.")
        break
    except openai.BadRequestError:
        print("That conversation got too long – forgetting the oldest messages.")
        history = [history[0]] + history[-4:]
```

Expected output on a normal run: each reply streams to the screen, and the conversation remembers earlier turns.

This version works correctly and meets every Build Task requirement. It's missing type hints and the structured-output feature — both worth adding next.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — everything in `main.py` 's loop, minimal structure

```
# chat_client.py
from openai import OpenAI

def create_client() -> OpenAI:
    return OpenAI()

def stream_message(client: OpenAI, history: list[dict], user_input: str) -> str:
    history.append({"role": "user", "content": user_input})
    stream = client.chat.completions.create(model="gpt-4o-mini", messages=history)
    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="")
            full_reply += piece
    print()
    history.append({"role": "assistant", "content": full_reply})
    return full_reply
```

```
# schemas.py
from pydantic import BaseModel

class ExtractedInfo(BaseModel):
    name: str
    intent: str
```

```
# main.py
import sys
import openai
from pydantic import ValidationError
from config import load_config
from logging_setup import get_logger
from chat_client import create_client, stream_message
```

```

from schemas import ExtractedInfo

def main() -> None:
    config = load_config()
    logger = get_logger(__name__)
    client = create_client()
    history: list[dict] = [{"role": "system", "content": "You are a helpful assistant"}]

    while True:
        user_input = input("You: ")
        if user_input == "/quit":
            break

        try:
            if user_input.startswith("/extract "):
                text = user_input.removeprefix("/extract ")
                try:
                    completion = client.beta.chat.completions.parse(
                        model="gpt-4o-mini",
                        messages=[{"role": "user", "content": text}],
                        response_format=ExtractedInfo,
                    )
                    print(completion.choices[0].message.parsed)
                except ValidationError:
                    print("Couldn't understand that clearly enough to extract structured info")
            else:
                stream_message(client, history, user_input)
        except openai.AuthenticationError:
            logger.error("Authentication failed – check OPENAI_API_KEY.")
            print("Login failed – check your API key.")
            sys.exit(1)
        except openai.BadRequestError:
            logger.warning("Conversation exceeded context length, trimming history")
            print("That got too long – forgetting the oldest messages and continuing")
            history = [history[0]] + history[-4:]

if __name__ == "__main__":
    main()

```

Expected output: the same streaming chat as Basic, plus typing `/extract Maria wants to reset her password` prints `ExtractedInfo(name='Maria', intent='reset her password')`, and `/extract the sky is blue` prints the clean fallback message.

Difference from Basic: full type hints throughout, a real `schemas.py` module, and the structured-output feature wired in behind its own `/extract` command with its own `except`

`ValidationError` — none of which Basic's version has yet.

Which one should you actually write? Intermediate's version already satisfies every Build Task requirement — full type hints, real memory, streaming, structured output, and all 3 graceful-failure cases. It's genuinely fine to ship as-is. A cached, explicitly-configured client (`OpenAI(max_retries=2, timeout=30.0)`), built once and reused) is worth adding once this code is meant to be imported and reused as a starting point for later projects — which is exactly what happens per this document's Goal — but it's a refinement, not a requirement.